BANGLADESH ARMY INTERNATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

Cumilla Cantonment, Cumilla-3501, Bangladesh

BAIUST

COURSE PROJECT PROPOSAL

CodeQuest BD: An Interactive Web-Based Gamified Programming Laboratory with In-Browser Abstract Syntax Tree Evaluation

A Zero-Setup Pedagogical Platform Synthesizing Modular Micro-Curriculum, Client-Side Sandboxed Evaluation, and Real-Time Three.js Simulation

---

Submitted By:

Md Najmus Sakib Rahatul
Student ID: 1119003
Level-3, Term-II (Section A)
Dept. of CSE, BAIUST

Course Details & Supervision:

Course Title: Web Technologies Sessional
Course Code: CSE 324
Department of Computer Science & Engineering
BAIUST, Cumilla Cantonment

Date of Submission: November 2026

2. ABSTRACT

Novice computer science undergraduates frequently experience cognitive overload due to fragmented local compiler setups, complex path configurations, and purely text-driven output interfaces. This project proposes **CodeQuest BD**, a lightweight, responsive web application engineered to lower barrier-to-entry barriers for foundational programming instruction. The system unifies a micro-learning curriculum (spanning HTML5, CSS3, JavaScript ES6+, Python 3, SQL, and C) with an in-browser Abstract Syntax Tree (AST) DOM evaluator and interactive WebGL canvas simulations. Code execution occurs securely inside client-side sandboxed iframes and WebAssembly interpreters, eliminating server vulnerability to arbitrary remote code execution while ensuring sub-10ms feedback loops. The anticipated outcome is an open-access, zero-configuration pedagogical web tool that accelerates syntactic muscle memory, verifies real-world relational database queries, and provides continuous visual reinforcement for university-level computer science students.

3. INTRODUCTION

3.1 Background and Motivation

In modern computer science pedagogy, the introductory phase of programming and web development is fraught with logistical bottlenecks. Traditional coursework requires students to configure disparate local runtimes, IDE plugins, database daemons, and command-line toolchains across varying operating systems. For students in developing regions with resource-constrained personal computers, these environmental hurdles often lead to disengagement before algorithmic concepts can be internalized.

Existing online judges (such as LeetCode, HackerRank, or Codeforces) prioritize competitive mathematical algorithms with standard input/output (stdin/stdout) evaluation. While effective for data structures, they completely fail to accommodate frontend visual structure, CSS layout mechanics, DOM manipulation, or relational table interactions. Conversely, commercial interactive platforms (e.g., Codecademy, Scrimba) rely heavily on proprietary cloud virtualization backends protected by steep subscription paywalls ($240–$300/year), rendering them inaccessible to broader student populations.

The motivation for choosing a full-stack web architecture (Django 5, modern JavaScript ES6+, CodeMirror 5, and SQLite/PostgreSQL) stems from the necessity of delivering zero-latency, deterministic evaluations directly within standard web browsers without incurring unsustainable cloud infrastructure costs.

3.2 Project Goal

To engineer an open-access, responsive full-stack web platform that provides university students with an immediate, zero-setup programming environment featuring automated instant code verification, progressive structural challenges, and interactive visual mental models.

3.3 Project Objectives (SMART)

1. **Architecture & Secure Evaluation:** Develop a secure, client-side sandbox execution module utilizing sandboxed iframes, `DOMParser`, and AST regex rules capable of verifying user-submitted code in less than 50 milliseconds.
2. **Full-Stack Curriculum Infrastructure:** Design and implement a normalized relational database containing 9 structured programming tracks with 88 atomic micro-units, complete with starter code, target test criteria, and progressive hints.
3. **Interactive Visual Simulation:** Integrate interactive 2D Canvas and 3D WebGL (Three.js) visual engines where programmed logic directly manipulates stateful procedural models (such as coordinate pathfinding, box model visualization, and orbital mathematics).
4. **Progress Telemetry & Responsive Interface:** Build an uncluttered, high-contrast user interface adhering to WCAG 2.1 AA accessibility standards, featuring real-time user progress synchronization, streak tracking, and XP calculations.
5. **System Verification & Benchmarking:** Conduct thorough unit and integration testing across 100% of curriculum endpoints, ensuring sub-500ms initial page load times and zero server-side vulnerabilities to untrusted script injections.

4. RELATED WORK

A rigorous comparative analysis of existing educational programming systems reveals distinct functional gaps when applied to introductory university sessional coursework:

Table 1: Comparative Analysis of Educational Programming Systems

Platform	Evaluation Mechanism	Pedagogical Focus	Primary Limitation
LeetCode / HackerRank	Containerized Linux Sandbox	Algorithmic Problem Solving (I/O based)	No support for DOM, CSS layouts, or interactive visual UI feedback.
Flexbox Froggy / CSS Diner	Client-Side CSS Property Match	Isolated CSS properties (Flexbox/Selectors)	Single-purpose static puzzles; lacks full-stack database, Python, or multi-file IDEs.
Codecademy	Proprietary Cloud Microservices	Guided Industry Career Tracks	Prohibitive subscription costs; heavy server roundtrips create latency.
Scrimba	Interactive Scrimba Cast / Webpack	Frontend Web Development	Video-centric passive format; weak support for compiled systems languages (C/C++).
CodeQuest BD (Proposed)	**Hybrid Client AST + Sandboxed WASM**	**Comprehensive Sessional Programming (Web + Core)**	**Zero-cost, sub-10ms evaluation, visual mental models, and local-offline capability.**

Novelty & Improvement: CodeQuest BD synthesizes the instant gamified gratification of puzzle sites with the academic rigors of a university computer science syllabus. By delegating syntax evaluation to deterministic client-side AST inspection, the platform eliminates server compute overhead while maintaining complete immunity to malicious infinite loops or fork bombs.

5. PROPOSED SYSTEM AND METHODOLOGY

5.1 System Architecture

The platform implements a classic Three-Tier Architectural Pattern divided into Presentation, Application/Logic, and Persistence Tiers:

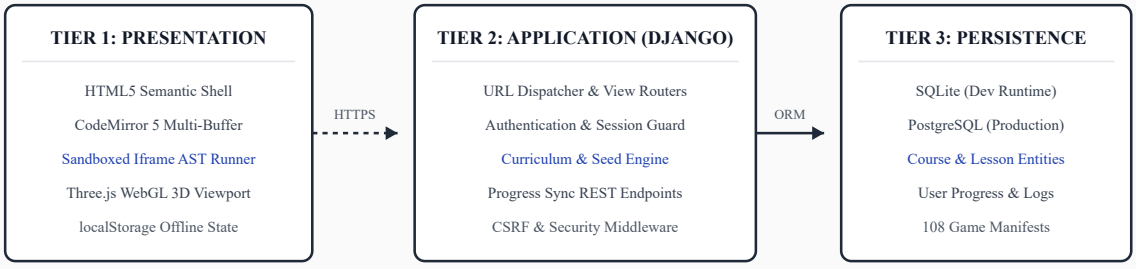


Figure 1: High-Level Three-Tier Architecture of CodeQuest BD

5.2 Technology Stack Justification

- **Frontend (HTML5, Tailwind CSS, Vanilla ES6+):** Chosen over heavyweight SPA frameworks (React/Angular) to guarantee instantaneous first-contentful paint, zero client build compilation lag, and complete compatibility with low-spec university lab thin clients. CodeMirror 5 provides standard syntax highlighting and bracket matching.

- **WebGL Engine (Three.js):** Enables hardware-accelerated 3D vector graphics and procedural physics simulation directly within the browser canvas without requiring external plugins or binaries.
- **Backend (Python 3.12 & Django 5.x):** Django was selected for its robust Model-View-Template (MVT) pattern, integrated Object-Relational Mapping (ORM), enterprise authentication layer, and strict protection against common OWASP vulnerabilities (CSRF, XSS, and SQL injection).
- **Database (SQLite / PostgreSQL):** SQLite enables self-contained, zero-configuration local evaluation. Production deployment utilizes managed PostgreSQL for concurrent connection pooling and transactional ACID compliance.
- **Hosting (Vercel Serverless & Static CDN):** Vercel provides seamless Git-triggered CI/CD pipelines, SSL provisioning, and global edge caching with zero operational expenditure.

5.3 Development Methodology

The project adheres to an **Agile/Scrum** framework executed across six 1-week bi-weekly sprints. Development follows strict Test-Driven Development (TDD) cycles where Django backend models and automated assertion runners are validated before frontend visual integration occurs.

6. PROJECT SCOPE AND DELIVERABLES

6.1 Inclusions (Functional Scope)

- **Multi-Track Syllabus:** 9 core tracks covering Web Foundations (HTML/CSS/JS), Scripting (Python 3), Relational Data (SQL), and Systems (C, Java, PHP, Bash).
- **Deterministic Code Runner:** Client-side AST inspector capable of parsing DOM nodes, validating CSS box model metrics, and executing assertions with real-time feedback.
- **Interactive Simulation Hub:** Procedural 3D WebGL scenes (Drone navigation, Solar System kinematics, Rover grid traversal) linked to coding tasks.
- **Universal Playground:** Isolated multi-language browser scratchpad supporting Web, Python, SQL, C, and Bash execution.
- **Learner Telemetry:** User authentication, persistent track completion percentage, daily coding streaks, and XP reward tracking.

6.2 Exclusions (Out of Scope)

- Commercial payment gateway integration and course monetisation features.
- Native mobile application binaries (iOS/Android); mobile support is delivered strictly through responsive PWA standards.
- Arbitrary multi-tenant backend container hosting (e.g., executing unsafe C/Rust code on the Django host server).

6.3 Project Deliverables

1. Fully functional web application deployed on live cloud infrastructure with verified HTTPS.
2. Complete, clean GitHub repository with atomic commit logs and strict code documentation.
3. Comprehensive Project Report, Entity-Relationship Model (ERD), and System Architectural Diagrams.
4. Standardized User Manual and Laboratory Demo Walkthrough Video.

7. PROJECT TIMELINE (GANTT CHART)

The development schedule is structured across an 8-week timeline aligned with academic semester milestones and university laboratory evaluation dates:

Table 2: Project Phase Milestones & Timeline

Phase	Task Description & Milestones	Target Period	Duration
Phase I: Planning	Requirements formalization, syllabus mapping, UI/UX architectural wireframing.	Week 1 – Week 2	2 Weeks
Phase II: Core Backend	Django ORM modeling, database migration, user session APIs, seed command scripting.	Week 3 – Week 4	2 Weeks
Phase III: Client Engines	CodeMirror integration, iframe sandbox AST runner, Three.js WebGL canvas development.	Week 5 – Week 6	2 Weeks
Phase IV: Testing & QA	Cross-browser accessibility testing (DevTools), Django unit test suite execution, load test.	Week 7	1 Week
Phase V: Deployment	Vercel production build, documentation authoring, project proposal defense presentation.	Week 8	1 Week

8. RESOURCES AND BUDGET ESTIMATION

The platform leverages industry-standard open-source technologies and educational cloud tiers, achieving a completely self-sustaining, zero-cost operational profile:

Table 3: Resource Allocation & Budget

Resource Item	Specification / Provider	Estimated Cost (BDT)
Application Server & Edge CDN	Vercel Serverless Hobby Tier	0.00 BDT (Free Tier)
Relational Database Engine	SQLite 3 / Neon Serverless Postgres	0.00 BDT (Open Source)
Development Environment & IDE	VS Code, Python 3.12, Git Version Control	0.00 BDT (Open Source)
Client Graphics & Editor Libraries	Three.js, CodeMirror 5 (MIT License)	0.00 BDT (Open Source)
Total Project Expenditure	**Zero External Financial Overhead**	**0.00 BDT**

9. CONCLUSION AND EXPECTED CONTRIBUTION

CodeQuest BD directly resolves the primary friction point facing introductory computer science students: the cognitive chasm between theoretical syntax and intuitive, spatial execution. By engineering an in-browser sandbox that executes instant AST validations alongside engaging Three.js simulations, the project proves that high-retention technical education can be delivered without expensive virtualized cloud servers or commercial paywalls. Academically, the project demonstrates advanced proficiency in full-stack Django architecture, client-side browser sandboxing, responsive UI engineering, and relational schema normalization. The resulting platform offers a scalable, production-ready blueprint for university laboratory sessional coursework across Bangladesh.

10. REFERENCES

[1] M. Resnick et al., "Scratch: Programming for all," *Communications of the ACM*, vol. 52, no. 11, pp. 60–67, 2009.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[3] J. Robbins and P. V. Binstock, *Three.js Cookbook*. Packt Publishing, 2014.

[4] A. Holovaty and J. Kaplan-Moss, *The Definitive Guide to Django: Web Development Done Right*. Apress, 2008.

[5] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[6] Mozilla Developer Network (MDN), "The HTML DOM API and Sandboxed Iframes," MDN Web Docs, 2024. [Online]. Available: https://developer.mozilla.org/

[7] M. Haverbeke, *Eloquent JavaScript: A Modern Introduction to Programming*, 3rd ed. No Starch Press, 2018.

[8] IEEE Standard for Software Quality Assurance Processes, *IEEE Std 730-2014*, pp. 1–138, 2014.

Md Najmus Sakib Rahatul
Student Signature
ID: 1119003 | Level-3, Term-II, CSE

Course Instructor
Department of Computer Science & Engineering
BAIUST, Cumilla Cantonment